

**Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Departamento de Ciências de Computação
Disciplina de Organização de Arquivos (SCC0215)**

Docente

Profa. Dra. Cristina Dutra de Aguiar

cdac@icmc.usp.br

Monitores

Gustavo Ramos Santos Pires

gustavo.rspires@usp.br ou telegram: @darrocaxd

João Gabriel Pieroli da Silva

joaogabrielpieroli@usp.br ou telegram: @bielcomp24

Pedro Lunkes Villela

pedrolunkesvillela@usp.br ou telegram: @pedrolunkes

Renan Banci Catarin

renanbcatarin@usp.br ou telegram: @Reckat

Renan Trofino Silva

renan.trofino@usp.br ou telegram: @renan823

Vinícius Souza Freitas

vininicius@usp.br ou telegram: @Vininicius

Trabalho Prático 1

Este trabalho tem como objetivo indexar arquivos de dados usando um índice árvore-B e utilizar esse índice para recuperar e inserir dados de um arquivo binário.

*O trabalho deve ser feito por, no máximo, **2 alunos da mesma turma**. Os alunos devem ser os mesmos do trabalho introdutório. Quaisquer mudanças devem ser devidamente informadas. A solução deve ser proposta exclusivamente pelo(s) aluno(s) com base nos conhecimentos adquiridos nas aulas. Consulte as notas de aula e o livro texto quando necessário.*

Descrição do arquivo de índice árvore-B

O índice árvore-B com ordem m é definido formalmente como descrito a seguir.

1. Cada página (ou nó) do índice árvore-B deve ser, pelo menos, da seguinte forma:

$\langle \langle C_1, P_{R1} \rangle, \langle C_2, P_{R2} \rangle, \dots, \langle C_{q-1}, P_{Rq-1} \rangle, P_1, P_2, \dots, P_{q-1}, P_q \rangle$, onde $(q \leq m)$ e

- Cada C_i ($1 \leq i \leq q-1$) é uma chave de busca.
- Cada P_{Ri} ($1 \leq i \leq q-1$) é um campo de referência para o registro no arquivo de dados que contém o registro de dados correspondente a C_i .

- Cada P_j ($1 \leq j \leq q$) é um campo de referência para uma subárvore ou assume o valor -1 caso não exista subárvore (ou seja, caso seja um nó folha).
2. Dentro de cada página (ou seja, as chaves de busca são ordenadas)
 - $C_1 < C_2 < \dots < C_{q-1}$.
 3. Para todos os valores X da chave na subárvore apontada por P_i :
 - $C_{i-1} < X < C_i$ para $1 < i < q$
 - $X < C_i$ para $i = 1$
 - $C_{i-1} < X$ para $i = q$.
 4. Cada página possui um máximo de m descendentes.
 5. Cada página, exceto a raiz e as folhas, possui no mínimo $\lceil m/2 \rceil$ descendentes (*taxa de ocupação*).
 6. A raiz possui pelo menos 2 descendentes, a menos que seja um nó folha.
 7. Todas as folhas aparecem no mesmo nível.
 8. Uma página não folha com k descendentes possui $k-1$ chaves.
 9. Uma página folha possui no mínimo $\lceil m/2 \rceil - 1$ chaves e no máximo $m - 1$ chaves (*taxa de ocupação*).

Descrição do Registro de Cabeçalho. O registro de cabeçalho deve conter os seguintes campos:

- *status*: indica a consistência do arquivo de índice, devido à queda de energia, travamento do programa, etc. Pode assumir os valores '0', para indicar que o arquivo de índice está inconsistente, ou '1', para indicar que o arquivo de índice está consistente. Ao se abrir um arquivo para escrita, seu *status* deve ser '0' e, ao finalizar o uso desse arquivo, seu *status* deve ser '1' – tamanho: *string* de 1 *byte*.
- *noRaiz*: armazena o RRN do nó (página) raiz do índice árvore-B. Quando a árvore-B está vazia, *noRaiz* = -1 – tamanho: inteiro de 4 *bytes*.
- *topo*: armazena o RRN de um registro logicamente removido, ou -1 caso não haja registros logicamente removidos – tamanho: inteiro de 4 *bytes*.
- *proxRRN*: armazena o valor do próximo RRN a ser usado para conter um nó (página da árvore-B). Inicialmente, a árvore-B está vazia e, portanto, *proxRRN* =

0. Quando o primeiro nó é criado (nó folha = nó raiz), $proxRRN = 1$. Depois, quando primeiro *split* acontece, $proxRRN = 3$. A cada nó criado da árvore-B, $proxRRN$ é incrementado – tamanho: inteiro de 4 bytes

- *nroNos*: armazena o número de nós (páginas) do índice árvore-B. Inicialmente, a árvore-B está vazia e, portanto, $nroNos = 0$. A cada novo nó inserido na árvore-B, $nroNos$ é incrementado e a cada nó removido da árvore-B, $nroNos$ é decrementado – tamanho: inteiro de 4 bytes

Representação Gráfica do Registro de Cabeçalho. O tamanho do registro de cabeçalho deve ser de 17 bytes, representado da seguinte forma:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1 byte <i>status</i>	4 bytes <i>noRaiz</i>				4 bytes <i>topo</i>				4 bytes <i>proxRRN</i>				4 bytes <i>nroNos</i>			

Observações Importantes.

- O registro de cabeçalho deve seguir estritamente a ordem definida na sua representação gráfica.
- Os nomes dos atributos também devem seguir estritamente os nomes definidos na especificação deles.
- Para seguir a especificação do conceito de árvore-B, o nó da árvore-B deve obrigatoriamente ser do tamanho de uma página de disco. Entretanto, isso não será seguido neste trabalho para simplificar a quantidade de chaves de busca que são armazenadas no nó. Lembrando também que as páginas de disco têm potência de 2, o que também não será seguido neste trabalho por simplificação.

Descrição do Registro de Dados. Deve ser considerada a seguinte organização: campos de tamanho fixo e registros de tamanho fixo. Em adição ao Item 1 da definição formal do índice árvore-B, cada nó (página) da árvore também deve armazenar os seguintes campos:

- *removido*: indica se o nó está logicamente removido. Pode assumir os valores ‘1’, para indicar que o nó está marcado como logicamente removido, ou ‘0’, para indicar que o nó não está marcado como removido. – tamanho: *string* de 1 byte.

- *próximo*: armazena o RRN do próximo nó logicamente removido – tamanho: inteiro de 4 bytes. Deve ser inicializado com o valor -1 quando necessário.
- *tipoNo*: indica o tipo de um nó, da seguinte forma: (i) *tipoNo* = -1 indica nó folha; (ii) *tipoNo* = 0 indica nó raiz; e (iii) *tipoNo* = 1 indica nó intermediário. Quando nó-folha = nó-raiz, *tipoNo* = -1 – tamanho: inteiro de 4 bytes.
- *nroChaves*, indicando o número de chaves presentes no nó – tamanho: inteiro de 4 bytes.

A ordem da árvore-B é 4, ou seja, $m = 4$. Portanto, um nó (página) terá 3 chaves no máximo e 4 descendentes. A chave de busca é o campo *codEstacao*. Lembrando que, em aplicações reais, a ordem da árvore-B é muito maior, para acomodar mais chaves. A proposta da árvore-B é que ela seja larga e baixa, para diminuir o número de acessos a disco.

Representação Gráfica de um Nó (Página/Registro de Dados) do índice. O tamanho dos registros de dados deve ser de 53 bytes, representado da seguinte forma:

0	1 ... 4	5 ... 8	9 ... 12	13 ... 16	17 ... 20	21 ... 24
1 byte	4 bytes	4 bytes	4 bytes	4 bytes	4 bytes	4 bytes
<i>removido</i>	<i>próximo</i>	<i>tipoNo</i>	<i>nroChaves</i>	C_1	P_{R1}	C_2

25 ... 28	29 ... 32	33 ... 36	37 ... 40	41 ... 44	45 ... 48	49 ... 52
4 bytes	4 bytes	4 bytes	4 bytes	4 bytes	4 bytes	4 bytes
P_{R2}	C_3	P_{R3}	P_1	P_2	P_3	P_4

Observações Importantes.

- Cada registro de dados deve seguir estritamente a ordem definida na sua representação gráfica.
- Os nomes dos atributos também devem seguir estritamente os nomes definidos na especificação deles.
- Quando um nó (página) do índice tiver chaves de busca que não forem preenchidas, a chave de busca deve ser representada pelo valor -1 e o ponteiro para o arquivo de dados deve ser representado pelo valor -1.
- O valor -1 deve ser usado para denotar que um ponteiro P_i ($1 \leq i \leq m$) de um nó da árvore-B é nulo.

Algoritmos de inserção e remoção. Os algoritmos de inserção e remoção devem ser implementados conforme descrito a seguir. Adicionalmente, deve ser feito o tratamento de nós (ou seja, páginas) logicamente removidos.

Detalhes sobre o algoritmo de inserção. Considere que deve ser implementada a rotina de *split* durante a inserção. Considere que a rotina de redistribuição durante a inserção não deve ser implementada (ou seja, não é uma árvore-B*). Considere que a distribuição das chaves de busca deve ser o mais uniforme possível, ou seja, considere que a chave de busca a ser promovida deve ser a chave que distribui o mais uniformemente possível as demais chaves entre o nó à esquerda e o novo nó resultante do particionamento. Quando necessário, considere que a chave de busca a ser promovida deve ser a primeira chave do novo nó resultante do particionamento (ou seja, o primeiro elemento do segundo nó é a chave promovida durante o particionamento). Adicionalmente, quando necessário, o nó mais à esquerda deverá conter uma chave de busca a mais. Considere que a página sendo criada é sempre a página à direita.

Detalhes sobre o algoritmo de remoção. Considere que a troca de uma chave que não está em um nó folha deve ser feita sempre com a sua sucessora imediata que está em um nó folha. Considere que, em caso de *underflow*, a redistribuição deve ser realizada considerando primeiro a página adjacente à direita. Se não for possível aplicá-la, a redistribuição deve ser realizada com a página adjacente à esquerda. Caso não seja possível realizar a redistribuição, então deve ser feita a concatenação, a qual deve ser realizada considerando a página adjacente à esquerda. Em caso de redistribuição, considere que a distribuição das chaves de busca deve ser o mais uniforme possível, ou seja, considere que a chave de busca a ser promovida deve ser a chave que distribui o mais uniformemente possível as demais chaves entre o nó à esquerda e o nó à direita. Quando necessário, considere que a chave de busca a ser promovida deve ser a primeira chave do nó da direita (ou seja, o primeiro elemento do nó à direita é a chave promovida durante a redistribuição). Adicionalmente, quando necessário, o nó mais à esquerda deverá conter uma chave de busca a mais. Em caso de concatenação, considere que todas as chaves que foram concatenadas devem ser armazenadas no nó à esquerda. Considere também que a página sendo destruída é sempre a página à direita.

Detalhes sobre o tratamento de nós logicamente removidos. Como uma das etapas da concatenação, uma página da árvore-B é liberada para ser posteriormente reutilizada. Neste projeto, o controle de páginas da árvore-B que foram liberadas deve ser feito seguindo a mesma matéria sobre *abordagem dinâmica* de reaproveitamento de espaços de registros logicamente removidos. A implementação dessa funcionalidade deve ser realizada usando o conceito de *pilha de registros logicamente removidos*, e deve seguir estritamente a matéria apresentada em sala de aula. Todas as vezes que uma página da árvore-B for liberada como resultado da rotina de *concatenação*, essa página deve ser marcada como logicamente removida e seu RRN deve ser empilhado, ou seja, os campos *topo* do registro de cabeçalho da árvore-B e *próximo* do registro de dados da árvore-B devem ser atualizados. Ao se remover uma página, os valores dos *bytes* referentes aos campos já armazenados na página devem permanecer os mesmos, com exceção dos valores dos campos relacionados ao tratamento da lista encadeada. Por outro lado, todas as vezes que uma nova página da árvore-B precisar ser criada como resultado da rotina de *split*, deve haver o reaproveitamento de uma página empilhada quando o valor do campo *topo* do registro de cabeçalho da árvore-B for diferente de -1. O lixo que permanece no registro logicamente removido e que não é reutilizado deve ser identificado pelo caractere '\$'.

Programa

Descrição Geral. Implemente um programa em C por meio do qual o usuário possa inserir dados em arquivos binários, bem como criar índices para indexar esses arquivos. Os índices são caracterizados por serem do tipo árvore-B.

Importante. A definição da sintaxe de cada comando bem como sua saída devem seguir estritamente as especificações definidas em cada funcionalidade. Para especificar a sintaxe de execução, considere que o programa seja chamado de “programaTrab”. Essas orientações devem ser seguidas uma vez que a correção do funcionamento do programa se dará de forma automática. De forma geral, a primeira entrada da entrada padrão é sempre o identificador de suas funcionalidades, conforme especificado a seguir.

Modularização. É importante modularizar o código. Trechos de programa que aparecerem várias vezes devem ser modularizados em funções e procedimentos.

Descrição Específica. O programa deve oferecer as seguintes funcionalidades:

Na linguagem SQL, o comando CREATE TABLE é usado para criar uma tabela, a qual é implementada como um arquivo. Geralmente, indica-se um campo (ou um conjunto de campos) que consiste na chave primária da tabela. Isso é realizado especificando-se a cláusula PRIMARY KEY. A funcionalidade [7] representa um exemplo de implementação de um índice árvore-B definido sobre o campo chave primária de um arquivo de dados.

Na linguagem SQL, o comando CREATE INDEX é usado para criar um índice sobre um campo (ou um conjunto de campos) de busca. A funcionalidade [7] representa um exemplo de implementação de um índice árvore-B definido sobre o campo chave primária de um arquivo de dados.

[7] Crie um arquivo de índice árvore-B para um arquivo de dados de entrada já existente, que é o arquivo de dados definido de acordo com a especificação do trabalho prático introdutório, e que pode conter registros logicamente removidos. O campo a ser indexado é *codEstacao*. Registros logicamente removidos presentes no arquivo de dados de entrada não devem ter suas chaves de busca correspondentes no arquivo de índice. A inserção no arquivo de índice deve ser feita um-a-um. Ou seja, para cada registro não removido presente no arquivo de dados, deve ser feita a inserção de sua chave de busca correspondente no arquivo de índice árvore-B. A manipulação do arquivo de índice árvore-B deve ser feita em disco, de acordo com o conteúdo ministrado em sala de aula. Antes de terminar a execução da funcionalidade, deve ser utilizada a função *binarioNaTela*, disponibilizada na página do projeto da disciplina, para mostrar a saída do arquivo de índice árvore-B.

Entrada do programa para a funcionalidade [7]:

7 arquivoEntrada.bin arquivoArvoreB.bin

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório.
- arquivoArvoreB.bin é um arquivo binário que indexa o arquivo de dados arquivoEntrada.bin e que é gerado conforme as especificações descritas neste trabalho prático.

Saída caso o programa seja executado com sucesso:

Listar o arquivo de índice no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab  
7 estacao.bin estacaoArvoreB.bin
```

usar a função binarioNaTela antes de terminar a execução da funcionalidade, para mostrar a saída do arquivo estacaoArvoreB.bin.

Na linguagem SQL, o comando SELECT é usado para listar os dados de uma tabela. Existem várias cláusulas que compõem o comando SELECT. Além das cláusulas SELECT e FROM, outra cláusula muito comum é a cláusula WHERE, que permite que seja definido um critério de busca sobre um ou mais campos, o qual é nomeado como critério de seleção.

SELECT lista de colunas (ou seja, campos a serem exibidos na resposta)

FROM tabela (ou seja, arquivo que contém os campos)

WHERE critério de seleção (ou seja, critério de busca)

A funcionalidade [8] representa um exemplo de implementação do comando SELECT considerando a cláusula WHERE. Como existe um índice árvore-B definido sobre o campo que representa a chave de busca, qualquer busca que utilize este campo deve ser feita com o auxílio do índice. Para as buscas que não utilizem o campo que representa a chave de busca, deve ser seguida a especificação da funcionalidade [3].

[8] Permita a recuperação dos dados de todos os registros de um arquivo de dados de entrada, de forma que esses registros satisfaçam um critério de busca determinado pelo usuário. Qualquer campo pode ser utilizado como forma de busca. Adicionalmente, a busca deve ser feita considerando um ou mais campos. Por exemplo, é possível realizar a busca considerando somente o campo *codEstacao* ou considerando os campos *nomeEstacao* e *nomeLinha*. Esta funcionalidade pode retornar 0 registros (quando nenhum satisfaz ao critério de busca), 1 registro (quando apenas um satisfaz ao critério de busca), ou vários registros. Como existe um índice árvore-B definido sobre o campo que representa a chave de busca, qualquer busca que utilize este campo deve ser feita com o auxílio do índice. Para as buscas que não utilizem o campo que representa a chave de busca, deve ser seguida a especificação da funcionalidade [3]. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas ("). Para a manipulação de *strings* com aspas duplas, pode-se usar a função `scan_quote_string` disponibilizada na página do projeto da disciplina. Para a busca por valores nulos, deve-se especificar o valor NULO. Registros marcados como logicamente removidos não devem ser exibidos. O arquivo de dados de entrada deve ser percorrido apropriadamente.

Sintaxe do comando para a funcionalidade [8]:

```
8 arquivoEntrada.bin arquivoArvoreB.bin n
m1 nomeCampo1 valorCampo1 ... nomeCampom1 valorCampom1
m2 nomeCampo1 valorCampo1 ... nomeCampom2 valorCampom2
...
mn nomeCampo1 valorCampo1 ... nomeCampomn valorCampomn
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório.

- arquivoArvoreB.bin é um arquivo binário que indexa o arquivo de dados arquivoEntrada.bin e que é gerado conforme as especificações descritas neste trabalho prático.

- n é a quantidade de vezes que a busca deve acontecer.

- m é a quantidade de vezes que o par nome do Campo e valor do Campo pode repetir em uma busca. Deve ser deixado um espaço em branco entre nomeCampo e valorCampo.

Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").

Saída caso o programa seja executado com sucesso:

Cada registro deve ser mostrado em uma única linha e os seus campos devem ser mostrados de forma sequencial separado por um espaço em branco. Campos de tamanho fixo que tiverem o valor nulo devem ser exibidos da seguinte forma: ao invés de exibir o valor -1, escreva NULO. Campos de tamanho variável que tiverem o valor nulo devem ser exibidos da seguinte forma: NULO. A ordem de exibição dos campos dos registros deve ser codEstacao, nomeEstacao, codLinha, nomeLinha, codProxEstacao, distProxEstacao, codLinhaIntegra, codEstIntegra. Ver exemplo ilustrado no **exemplo de execução**.

Mensagem de saída caso não seja encontrado o registro que contém o valor do campo ou o campo pertence a um registro que esteja removido:

Registro inexistente.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
8 estacao.bin estacaoArvoreB.bin 1
1 nomeEstacao "Luz"
9 Luz 1 Azul 10 NULO 4 55
55 Luz 4 Amarela 56 1257 1 9
```

111 Luz 7 Rubi 112 NULO NULO NULO

...

Na linguagem SQL, o comando DELETE é usado para remover dados em uma tabela. Para tanto, devem ser especificados quais dados (ou seja, registros) devem ser removidos, de acordo com algum critério.

DELETE FROM tabela (ou seja, arquivo que contém os campos)

WHERE critério de seleção (ou seja, critério de busca)

A funcionalidade [9] representa um exemplo de implementação do comando DELETE.

[9] Permita a remoção lógica de registros de um arquivo de dados de entrada, baseado na *abordagem dinâmica* de reaproveitamento de espaços de registros logicamente removidos. A implementação dessa funcionalidade deve ser realizada usando o conceito de *pilha de registros logicamente removidos*, e deve seguir estritamente a matéria apresentada em sala de aula. Os registros a serem removidos devem ser aqueles que satisfaçam um critério de busca determinado pelo usuário, sendo que a busca deve ser realizada conforme a especificação da funcionalidade [8]. Note que qualquer campo pode ser utilizado como forma de remoção. Ao se remover um registro, os valores dos *bytes* referentes aos campos já armazenados no registro devem permanecer os mesmos, com exceção dos valores dos campos relacionados ao tratamento da lista encadeada. Adicionalmente, as chaves de busca referentes aos registros logicamente removidos devem ser removidos do arquivo de índice da árvore-B criado na funcionalidade [7]. A funcionalidade [9] deve ser executada n vezes seguidas. Em situações nas quais um determinado critério de busca não seja satisfeito, ou seja, caso a solicitação do usuário não retorne nenhum registro a ser removido, o programa deve continuar a executar as remoções até completar as n vezes seguidas. Antes de terminar a execução da funcionalidade, deve ser utilizada a função `binarioNaTela`, disponibilizada na página do projeto da disciplina, para mostrar a saída dos arquivos binários de dados e de índice.

Entrada do programa para a funcionalidade [9]:

```
9 arquivoEntrada.bin arquivoArvoreB.bin n
m1 nomeCampo1 valorCampo1 ... nomeCampom1 valorCampom1
m2 nomeCampo1 valorCampo1 ... nomeCampom2 valorCampom2
...
mn nomeCampo1 valorCampo1 ... nomeCampomn valorCampomn
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório. As remoções a serem realizadas nessa funcionalidade devem ser feitas nesse arquivo.
- arquivoArvoreB.bin é um arquivo binário que indexa o arquivo de dados arquivoEntrada.bin e que é gerado conforme as especificações descritas neste trabalho prático.
- n é o número de remoções a serem realizadas.
- m é a quantidade de vezes que o par nome do Campo e valor do Campo pode repetir na busca pelos registros a serem removidos. Deve ser deixado um espaço em branco entre o nome do campo e o valor do campo. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").

Saída caso o programa seja executado com sucesso:

Listar o arquivo de dados e o arquivo de índice no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
9 estacao.bin estacaoArvoreB.bin 2
1 nomeEstacao "Luz"
2 nomeLinha "Verde" codProxEstacao 27
```

usar a função binarioNaTela antes de terminar a execução da funcionalidade, para mostrar a saída dos arquivos estacao.bin e estacaoArvoreB.bin, os quais foram atualizados frente às remoções.

Na linguagem SQL, o comando INSERT INTO é usado para inserir dados em uma tabela. Para tanto, devem ser especificados os valores a serem armazenados em cada coluna da tabela, de acordo com o tipo de dado definido. A funcionalidade [9] representa exemplo de implementação do comando INSERT INTO.

[10] Permita a inserção de novos registros em um arquivo de dados de entrada, baseado na *abordagem dinâmica* de reaproveitamento de espaços de registros logicamente removidos. A implementação dessa funcionalidade deve ser realizada usando o conceito de *pilha de registros logicamente removidos*, e deve seguir estritamente a matéria apresentada em sala de aula. O lixo que permanece no registro logicamente removido e que não é reutilizado deve ser identificado pelo caractere '\$'. Adicionalmente, as chaves de busca referentes aos novos registros devem ser inseridas no arquivo de índice da árvore-B criado na funcionalidade [7]. Na entrada da funcionalidade [10], os dados são referentes aos seguintes campos, na seguinte ordem: codEstacao, nomeEstacao, codLinha, nomeLinha, codProxEstacao, distProxEstacao, codLinhaIntegra, codEstIntegra. Campos com valores nulos, na entrada da funcionalidade, devem ser identificados com NULO. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas ("). Para a manipulação de *strings* com aspas duplas, pode-se usar a função `scan_quote_string` disponibilizada na página do projeto da disciplina. A funcionalidade [10] deve ser executada *n* vezes seguidas. Antes de terminar a execução da funcionalidade, deve ser utilizada a função `binarioNaTela`, disponibilizada na página do projeto da disciplina, para mostrar a saída dos arquivos binários (de dados e de índice).

Entrada do programa para a funcionalidade [10]:

```
10 arquivoEntrada.bin arquivoArvoreB.bin n
codEstacao1 nomeEstacao1 codLinha1 nomeLinha1 codProxEstacao1
distProxEstacao1 codLinhaIntegra1 codEstacaoIntegra1
codEstacao2 nomeEstacao2 codLinha2 nomeLinha2 codProxEstacao2
distProxEstacao2 codLinhaIntegra2 codEstacaoIntegra2
...
codEstacaon nomeEstacaon codLinhan nomeLinhan codProxEstacaon
distProxEstacaon codLinhaIntegran codEstacaoIntegran
```

onde:

- arquivoEntrada.bin é o arquivo binário que foi gerado conforme as especificações descritas no trabalho prático introdutório. As inserções a serem realizadas nessa funcionalidade devem ser feitas nesse arquivo.
- arquivoArvoreB.bin é um arquivo binário que indexa o arquivo de dados arquivoEntrada.bin e que foi gerado conforme as especificações descritas neste trabalho prático. As inserções realizadas no arquivo de dados devem ser refletidas nesse arquivo de índice.
- n é o número de inserções a serem realizadas. Para cada inserção, deve ser informado os valores a serem inseridos no arquivo, considerando os seguintes campos, na seguinte ordem: codEstacao, nomeEstacao, codLinha, nomeLinha, codProxEstacao, distProxEstacao, codLinhaIntegra, codEstIntegra. Valores nulos devem ser identificados, na entrada da funcionalidade, por NULO. Cada uma das n inserções deve ser especificada em uma linha diferente. Deve ser deixado um espaço em branco entre os valores dos campos. Os valores dos campos do tipo *string* devem ser especificados entre aspas duplas (").

Saída caso o programa seja executado com sucesso:

Listar o arquivo de dados e o arquivo de índice no formato binário usando a função fornecida binarioNaTela.

Mensagem de saída caso algum erro seja encontrado:

Falha no processamento do arquivo.

Exemplo de execução:

```
./programaTrab
10 estacao.bin estacaoArvoreB.bin 2
500 "Teste" 10 "Branca" NULO NULO NULO NULO
501 "Nova Estacao" 10 "Branca" NULO NULO NULO NULO
```


usar a função `binarioNaTela` antes de terminar a execução da funcionalidade, para mostrar a saída dos arquivos `estacao.bin` e `estacaoArvoreB.bin`, os quais foram atualizados frente às inserções.

Restrições

As seguintes restrições têm que ser garantidas no desenvolvimento do trabalho.

[1] O arquivo de dados deve ser gravado em disco no **modo binário**. O modo texto não pode ser usado.

[2] Os dados do registro descrevem os nomes dos campos, os quais não podem ser alterados. Ademais, todos os campos devem estar presentes na implementação, e nenhum campo adicional pode ser incluído. O tamanho e a ordem de cada campo deve obrigatoriamente seguir a especificação.

[3] Deve haver a manipulação de valores nulos, conforme as instruções definidas.

[4] Não é necessário realizar o tratamento de truncamento de dados.

[5] Devem ser exibidos avisos ou mensagens de erro de acordo com a especificação de cada funcionalidade.

[6] Os dados devem ser obrigatoriamente escritos campo a campo. Ou seja, não é possível escrever os dados registro a registro. Essa restrição refere-se à entrada/saída, ou seja, à forma como os dados são escritos no arquivo.

[7] O(s) aluno(s) que desenvolveu(desenvolveram) o trabalho prático deve(m) constar como comentário no início do código (i.e. NUSP e nome do aluno). Para trabalhos desenvolvidos por mais do que um aluno, não será atribuída nota ao aluno cujos dados não constarem no código fonte.

[8] Todo código fonte deve ser documentado. A **documentação interna** inclui, dentre outros, a documentação de procedimentos, de funções, de variáveis, de partes do código fonte que realizam tarefas específicas. Ou seja, o código fonte deve ser documentado tanto em nível de rotinas quanto em nível de variáveis e blocos funcionais.

[9] A implementação deve ser realizada usando a linguagem de programação C. As funções das bibliotecas `<stdio.h>` devem ser utilizadas para operações relacionadas à escrita e leitura dos arquivos. A implementação não pode ser feita em qualquer outra linguagem de programação. O programa executará no `[run.codes]`.

Material para Entregar

Arquivo compactado (a ser entregue no *run.codes*)

Deve ser preparado um arquivo .zip contendo:

- Código fonte do programa devidamente documentado.
- Makefile para a compilação do programa.

Vídeo (a ser entregue no *e-disciplinas*)

- Um vídeo gravado pelos integrantes do grupo, o qual deve ter, no máximo, 7 minutos de gravação. O vídeo deve explicar o trabalho desenvolvido. Ou seja, o grupo deve apresentar: cada funcionalidade e uma breve descrição de como a funcionalidade foi implementada. Todos os integrantes do grupo devem participar do vídeo, sendo que o tempo de apresentação dos integrantes deve ser balanceado. Ou seja, o tempo de participação de cada integrante deve ser aproximadamente o mesmo. O uso da webcam é obrigatório.

Instruções para fazer o arquivo makefile. No `[run.codes]` tem uma orientação para que, no makefile, a diretiva “all” contenha apenas o comando para compilar seu programa e, na diretiva “run”, apenas o comando para executá-lo. Adicionalmente, para

utilizar a função `binarioNaTela`, é necessário usar a flag `-lmd`. Assim, a forma mais simples de se fazer o arquivo `makefile` é:

```
all:
    gcc -o programaTrab *.c -lmd
run:
    ./programaTrab
```

Lembrando que `*.c` já engloba todos os arquivos `.c` presentes no seu zip. Adicionalmente, no arquivo `Makefile` é importante se ter um `tab` nos locais colocados acima, senão ele pode não funcionar.

Instruções de entrega.

O programa deve ser submetido via [run.codes]:

- página: <https://runcodes.icmc.usp.br/>
- **Segunda Feira:** código de matrícula: **TZQ3**
- **Terça Feira:** código de matrícula: **71W4**

O vídeo gravado deve ser submetido por meio da página da disciplina no e-disciplinas, no qual o grupo vai informar o nome de cada integrante, o número do grupo e um link que contém o vídeo gravado. Ao submeter o link, verifique se o mesmo pode ser acessado. Vídeos cujos links não puderem ser acessados receberão nota zero. Vídeos corrompidos ou que não puderem ser corretamente acessados receberão nota zero.

Critério de Correção

Critério de avaliação do trabalho. Na correção do trabalho, serão ponderados os seguintes aspectos.

- Corretude da execução do programa.
- Atendimento às especificações do registro de cabeçalho e dos registros de dados.
- Atendimento às especificações da sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade.

- Qualidade da documentação entregue. A documentação interna terá um peso considerável no trabalho.
- Estruturação da solução. O projeto deve apresentar uma organização adequada de arquivos e diretórios.
- Modularização. Trechos de código que aparecem repetidamente devem ser modularizados por meio de funções ou procedimentos, evitando duplicação de código.
- Uso de ferramentas de inteligência artificial. Não deve ser realizado para a implementação das funcionalidades especificadas neste trabalho e outras funções julgadas como necessárias para o embasamento do conteúdo ministrado. O uso dessas ferramentas acarretará desconto rigoroso na nota.
- Vídeo. Integrantes que não participarem da apresentação receberão nota 0 no trabalho correspondente.

Casos de teste no [run.codes]. Juntamente com a especificação do trabalho, serão disponibilizados 70% dos casos de teste no [run.codes], para que os alunos possam avaliar o programa sendo desenvolvido. Os 30% restantes dos casos de teste serão utilizados nas correções.

Restrições adicionais sobre o critério de correção.

- A não execução de um programa devido a erros de compilação implica que a nota final da parte do trabalho será igual a zero (0).
- O não atendimento às especificações do registro de cabeçalho e dos registros de dados implica que haverá uma diminuição expressiva na nota do trabalho.
- O não atendimento às especificações de sintaxe dos comandos de cada funcionalidade e do formato de saída da execução de cada funcionalidade implica que haverá uma diminuição expressiva na nota do trabalho.
- A ausência da documentação implica que haverá uma diminuição expressiva na nota do trabalho.
- A realização do trabalho prático com alunos de turmas diferentes implica que haverá uma diminuição expressiva na nota do trabalho.

- A inserção de palavras ofensivas nos arquivos e em qualquer outro material entregue implica que a nota final da parte do trabalho será igual a zero (0).
- Em caso de plágio, as notas dos trabalhos envolvidos serão zero (0).

Bom Trabalho!